Class06: R Functions

AUTHOR
May Alsoufi (PID: A17423250)

Background

Functions are at the heart of using R. Everything we do involves calling and using functions (from data input, analysis to results output).

All functions in R have at least 3 things:

1. A **name** the thing we use to call the function.
2. One or more input *arguments** that are comma separated.
3. The **body**, lines of code between curly brackets { } that does the work of the function.

A first function

Let's write a silly wee function to add some numbers:

```
add <-function(x){
  x+1
}
```

Let's try it out

```
add(100)
```

```
[1] 101
```

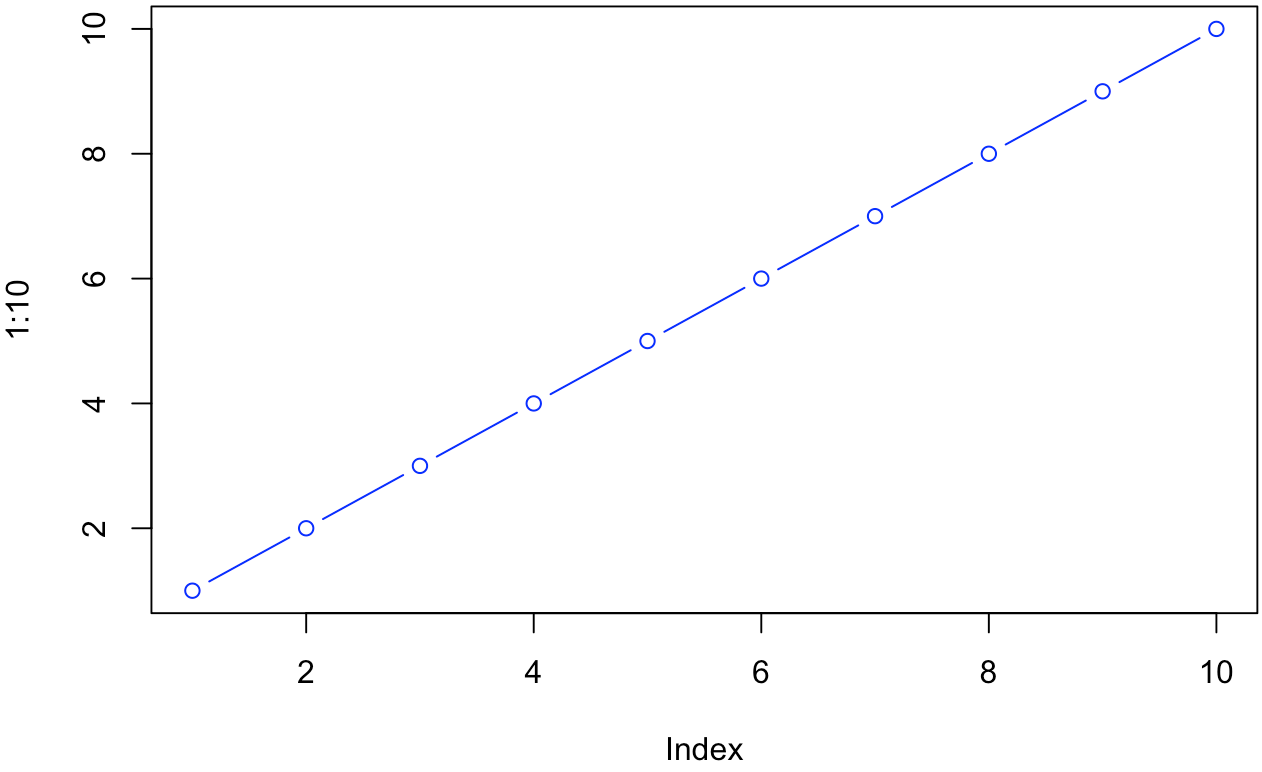
```
add <-function(x,y){
  x+y
}
```

Modify to be more useful and add more than just 1

```
add (100,10)
```

```
[1] 110
```

```
plot(1:10, col="blue", typ="b")
```




```
log(10,base=10)
```

```
[1] 1
```

> **N.M.** Input arguments can be either **required** or **optional**. The later have a fall-back default that is specified in the function code with an equals sign.

```
#add(x=100, y=200, z=300)
```

A second function

All functions in R look like this

```
name <--function(arg) {
  body
}
```

The `sample()` function in R is used to generate a random sample from a given vector or data set

```
sample(1:10, size=4)
```

```
[1] 2 7 3 4
```

> Q. Return 12 numbers picked randomly from input 1:10

```
sample(1:10, size=12, replace=TRUE)
```

```
[1]  8  9  7  5  5 10  9  6  3  3  3  5
```

> Q. Write the code to generate a random 12 nucleotide long DNA sequence?

```
bases<- c("A", "G", "C", "T")
sample(bases, size=12, replace=TRUE)
```

```
[1] "G" "G" "G" "C" "C" "C" "G" "A" "T" "G" "C" "A"
```

> Q. Write a first version function called `generate_dna()` that generates a user specified leangth `n` randome DNA sequence

```
generate_data <-function(n) {
  bases <- c("A", "G", "C", "T")
  sample(bases, size=n, replace=TRUE)
}
```

```
generate_data(10)
```

```
[1] "A" "C" "C" "T" "C" "T" "A" "T" "G" "G"
```

> Q. Modify your function to return a FASTA ike sequence rather than " [1] "G" "T" "C" "G" "C"", we want"GTCGC"

```
generate_data <-function(n) {
  bases <- c("A", "G", "C", "T")
   ans<-sample(bases, size = n, replace = TRUE)
   ans1<-paste(ans, collapse="")
   return(ans1)
}
```

```
generate_data(10)
```

```
[1] "AAATTCTTTT"
```

> Q. Give the user an option to return FASTA format output sequence or standard multi-element vector format

```
generate_data <-function(n, fasta=TRUE) {
  bases <- c("A", "G", "C", "T")
   ans<-sample(bases, size = n, replace = TRUE)

   if(fasta){
  ans<- paste(ans, collapse="")
   }

   return(ans)
}
```

```
generate_data(10)
```

```
[1] "GTCAAAACAA"
```

```
generate_data(10, fasta=F)
```

```
 [1] "T" "A" "T" "C" "A" "G" "C" "G" "A" "A"
```

A new cool function

> Q. Write a function called `generate_protein()` that generates a user specified length protein sequence in FASTA like format

```
generate_protein <- function(n, fasta=TRUE) {
  aa<- c("A","R","N", "D", "C", "Q", "E", "G", "H", "I", "L", "K", "M", "F", "P", "S", "T
  ans <- sample(aa, size = n, replace = TRUE)

  if (fasta) {
    ans <- paste(ans, collapse = "")
  }

  return(ans)
}
```

```
generate_protein(20)
```

```
[1] "IADCSCMPDFEETVTVFDNK"
```

> Q. Use your new `generate_protein()` function to generate all sequences between length 6 and 12 amino-acids in length and check if any of these are unique in nature (i.e. found in the NR database at NCBI).

```r
for(i in 6:12){
  cat(">", i, "\n", sep="")
  cat(generate_protein(i), "\n")
}
```

```
>6
SVPGAY
>7
QAQWWMR
>8
RHRKHPTP
>9
AQCYAWMTY
>10
GNGTYMCFYY
>11
HPEACFNWKAW
>12
NTFDKATDVLVT
```

ANS: Some of these sequences generated are found in nature. NCBI “pblast” shows 100% identity and 100% coverage for some of these sequences. However, after the 8 amino acid sequence, the sequences start being completely unique.